

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 4

1 - Enunciado

Implemente dois programas paralelos em C com OpenMP: um limitado por memória, com somas simples em vetores, e outro limitado por CPU, com cálculos matemáticos intensivos. Paralelize com `#pragma omp parallel for` e meça o tempo de execução variando o número de threads. Análise quando o desempenho melhora, estabiliza ou piora, e reflita sobre como o 'multithreading' de hardware pode ajudar em programas memory-bound, mas atrapalhar em programas compute-bound pela competição por recursos.

2 - Metodologia

O primeiro passo foi desenvolver 2 código separados e logo após compilar os mesmo sem usar nenhum tipo de otimização utilizando o seguinte comando:

```
gcc -fopenmp progmem.c -o progmem
```

```
gcc -fopenmp progcpu.c -o progcpu -lm
```

O “-lm” serve para vincular a biblioteca matemática padrão (libm) ao progcpu.c

Aqui está o desmembramento do símbolo:

- -l: Instructs (instrui) o compilador a linkar uma biblioteca.
- m: Indica o nome da biblioteca, que neste caso é math (matemática).

Sem isso o compilador apresenta o seguinte erro “error: undefined reference to ‘sin’”, que varia mediante a função usada seja ela trigonometria, logaritmo, e etc.

Além disso foi utilizado “omp_get_max_threads()” da biblioteca “omp.h” para medir a quantidade de threads usados durante a execução do código.

2.1 - Algoritmo

2.1.1 - progmem.c

1. Criar os vetores X, Y e Z.
2. Preencher X com 1 e Y com 2.
3. Iniciar a contagem do tempo.
4. Somar $X[i] + Y[i]$ e guardar em $Z[i]$ usando OpenMP.
5. Finalizar a contagem do tempo.
6. Mostrar o tempo de execução e o número de threads.
7. Liberar os vetores.

2.1.2 - progmem.c

1. Criar os vetores X, Y e Z.
2. Preencher X e Y com seus respectivos índices.
3. Iniciar a contagem do tempo.
4. Para cada posição de X, calcular $\sin(X[i]) * \cos(Y[j])$ 200 vezes.
5. Somar os resultados e guardar em $Z[i]$ usando OpenMP.
6. Finalizar a contagem do tempo.
7. Mostrar o tempo de execução e o número de threads.
8. Liberar os vetores.

2.3 - Ambiente de Execução

Componente	Especificação
Notebook	Lenovo IdeaPad 3 15ALC6
SO	Ubuntu 26.04 LTS
Kernel	Linux 7.0.0-29-generic
CPU	AMD Ryzen 5 5500U — 6C/12T
Arquitetura	x86_64
RAM	9,05 GiB

Desktop	GNOME 50.1
Shell	Zsh 5.9
Terminal	Kitty 0.45.0
Compilador	GCC

3 - Código Fonte

3.1 - Progpcu.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <math.h>

#define TAM 1000000
#define ITER 200

int main(void){

    double *X = malloc(TAM * sizeof(double));
    double *Y = malloc(TAM * sizeof(double));
    double *Z = malloc(TAM * sizeof(double));

    for(int i = 0; i < TAM; i++){
        X[i] = (double)i;
    }

    for(int j =0; j < TAM; j++){
        Y[j] = (double)j;
    }
```

```

double inicio= omp_get_wtime();

#pragma omp parallel for
for(int i = 0; i < TAM; i++){
    double acumulador = 0.0;
    for(int j = 0; j < ITER; j++){
        acumulador += (sin(X[i]) * cos(Y[j])) + sqrt(X[i] + Y[j]);
    }
    Z[i] = acumulador;
}

double fim = omp_get_wtime();

printf("Tempo progcpu.c: %f (s) | threads=%d\n", fim - inicio, omp_get_max_threads());

free(X);
free(Y);
free(Z);

return 0;
}

```

3.2 - progmem.c

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <math.h>

#define TAM 100000000

```

```
int main(void){

    double *X = malloc(TAM * sizeof(double));

    double *Y = malloc(TAM * sizeof(double));

    double *Z = malloc(TAM * sizeof(double));


    for(int i = 0; i < TAM; i++){

        X[i] = 1.0;

        Y[i] = 2.0;

    }


    double inicio = omp_get_wtime();

    #pragma omp parallel for
    for(int i = 0; i < TAM; i++){

        Z[i] = X[i] + Y[i];

    }

    double fim = omp_get_wtime();


    double soma = 0.0;

    for(int i = 0; i < TAM; i++){

        soma += Z[i];

    }


    printf("Tempo progcpu.c: %f (s) | threads=%d | Valor da soma(Z)=%e\n", fim - inicio,
    omp_get_max_threads(), soma);


    free(X);

    free(Y);

    free(Z);

}
```

```
return 0;
}
```

4 - Resultado

Para o conjunto de teste foi usado uma variação na quantidade de threads de 1 a 14, porém o processador usado nos teste que é descrito no item 2.3, possui apenas 12 threads físicos.

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_4
> export OMP_NUM_THREADS=1; time ./progmem
export OMP_NUM_THREADS=2; time ./progmem
export OMP_NUM_THREADS=4; time ./progmem
export OMP_NUM_THREADS=6; time ./progmem
export OMP_NUM_THREADS=8; time ./progmem
export OMP_NUM_THREADS=10; time ./progmem
export OMP_NUM_THREADS=12; time ./progmem
export OMP_NUM_THREADS=14; time ./progmem
Tempo progcpu.c: 1.305193 (s) | threads=1 | Valor da soma(Z)=3.000000e+08
./progmem 1,57s user 1,87s system 99% cpu 3,446 total
Tempo progcpu.c: 0.665599 (s) | threads=2 | Valor da soma(Z)=3.000000e+08
./progmem 1,62s user 1,79s system 124% cpu 2,740 total
Tempo progcpu.c: 0.336057 (s) | threads=4 | Valor da soma(Z)=3.000000e+08
./progmem 1,63s user 1,82s system 142% cpu 2,418 total
Tempo progcpu.c: 0.199106 (s) | threads=6 | Valor da soma(Z)=3.000000e+08
./progmem 1,32s user 1,62s system 152% cpu 1,925 total
Tempo progcpu.c: 0.189774 (s) | threads=8 | Valor da soma(Z)=3.000000e+08
./progmem 1,25s user 1,57s system 189% cpu 1,483 total
Tempo progcpu.c: 0.185165 (s) | threads=10 | Valor da soma(Z)=3.000000e+08
./progmem 1,46s user 1,80s system 202% cpu 1,608 total
Tempo progcpu.c: 0.186017 (s) | threads=12 | Valor da soma(Z)=3.000000e+08
./progmem 1,50s user 1,95s system 233% cpu 1,479 total
Tempo progcpu.c: 0.183420 (s) | threads=14 | Valor da soma(Z)=3.000000e+08
./progmem 1,34s user 1,76s system 209% cpu 1,478 total
~/Doc/g/Programacao_Paralela/Tarefa_4 on main !2
```

Figura 1: Teste usando progmem

```

> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_4
) export OMP_NUM_THREADS=1; time ./progcpu
export OMP_NUM_THREADS=2; time ./progcpu
export OMP_NUM_THREADS=4; time ./progcpu
export OMP_NUM_THREADS=6; time ./progcpu
export OMP_NUM_THREADS=8; time ./progcpu
export OMP_NUM_THREADS=10; time ./progcpu
export OMP_NUM_THREADS=12; time ./progcpu
export OMP_NUM_THREADS=14; time ./progcpu
Tempo progcpu.c: 8.793561 (s) | threads=1
./progcpu 8,80s user 0,03s system 99% cpu 8,825 total
Tempo progcpu.c: 3.388138 (s) | threads=2
./progcpu 6,78s user 0,02s system 199% cpu 3,410 total
Tempo progcpu.c: 1.454216 (s) | threads=4
./progcpu 5,80s user 0,02s system 395% cpu 1,469 total
Tempo progcpu.c: 1.295341 (s) | threads=6
./progcpu 6,93s user 0,02s system 530% cpu 1,311 total
Tempo progcpu.c: 1.137473 (s) | threads=8
./progcpu 8,08s user 0,03s system 694% cpu 1,167 total
Tempo progcpu.c: 1.045820 (s) | threads=10
./progcpu 9,76s user 0,03s system 913% cpu 1,072 total
Tempo progcpu.c: 0.928982 (s) | threads=12
./progcpu 10,84s user 0,02s system 1148% cpu 0,945 total
Tempo progcpu.c: 0.968305 (s) | threads=14
./progcpu 10,41s user 0,02s system 1059% cpu 0,985 total

```

Figura 2: Teste usando progcpu

Após os testes realizados foi montada uma tabela com os dados dos testes lado a lado, na tabela foi adicionado o speedup que será calculado de acordo com a fórmula da figura 3:

$$\text{Speedup}(N) = \frac{T(1)}{T(N)}$$

Figura 3: Fórmula Speedup

Threads	progmem (WT -(s))	progmem (time - (s))	progcpu (WT - (s))	progcpu (time- ((s))	Speedup progmem	Speedup progcpu
1	1,305193	3,446	8,793561	8,825	1	1
2	0,665599	2,74	3,388138	3,41	1,96	2,6
4	0,336057	2,418	1,454216	1,469	3,88	6,05
6	0,199106	1,925	1,295341	1,311	6,56	6,79
8	0,189774	1,483	1,137473	1,167	6,88	7,73

10	0,185165	1,608	1,04582	1,072	7,05	8,41
12	0,186017	1,479	0,928982	0,945	7,02	9,47
14	0,18342	1,478	0,968305	0,985	7,12	9,08

Tabela 1: Resultados dos testes

5 - Conclusão

Os testes realizados permitiram observar de forma prática o comportamento de aplicações paralelas com diferentes quantidades de threads para uma mesma aplicação e o seu comportamento diante de gargalos que podem ser encontrados em aplicações reais.

No programa progmem.c, caracterizado por operações simples de soma em vetores, houve um ganho de desempenho com o aumento do número de threads como já era esperado pelo que foi discutido em sala, os ganhos são mais visíveis até a 8 threads. A partir desse ponto, o speedup apresentou pouca evolução, passando de 6,88 com 8 threads para 7,12 com 14 threads. Esse comportamento mostra que, a partir do momento em que o acesso à memória se torna o principal gargalo durante a execução, aumentar a quantidade de threads deixa de proporcionar um aumento de desempenho que seja proporcional aos recursos gastos. Isso ocorre pois, com a grande quantidade de dados sendo processada, o uso da memória principal (RAM) se torna necessário pois a memória cache é insuficiente para manter todos os dados necessários ao processamento. Dessa forma, as threads passam a competir pelo acesso aos recursos de memória, limitando os ganhos obtidos com o paralelismo.

Já o progcpu.c, composto por cálculos matemáticos mais intensivos, apresentou uma melhoria consistente com o aumento do número de threads. O speedup chegou a 9,47 com 12 threads, que corresponde à quantidade de threads de hardware usados. Entretanto, ao utilizar 14 threads, o speedup caiu para 9,08, o que demonstrando que ao ultrapassar a capacidade de hardware utilizando mais threads não necessariamente melhora o desempenho, que foi algo discutido na sala com isso confirmamos que a quantidade de threads ideal depende das características da aplicação e do hardware utilizados pois cada hardware e aplicação vai ter comportamentos diferentes. Portanto, os experimentos demonstram a importância de analisar o tipo de gargalo de uma aplicação antes de definir a quantidade de threads utilizada.